NCCLPol: Verified, Composable Policy Execution for GPU Collective Communication

Anonymous Author(s)

ABSTRACT

NCCL is the de facto standard library for GPU collective communication in large-scale distributed training. Its plugin system allows operators to customize collective behavior, but plugins execute as unverified native code: a null-pointer dereference crashes the training job; plugins share no state across extension points; and updating a plugin requires restarting the job. We observe that NCCL's CPU-side decision path is latency-tolerant—the GPU collective it configures takes microseconds to milliseconds—making eBPF's verification overhead negligible in exchange for safety, composability, and live update. We present NCCLPol, which embeds a userspace eBPF runtime inside NCCL's plugin interface, requiring zero NCCL modifications, to provide (1) load-time verification that catches crash-inducing bugs before execution, (2) typed eBPF maps for cross-plugin state sharing, and (3) atomic policy hot-reload without call loss. Our evaluation on NCCL 2.29.7 shows 52–100 ns of CPU overhead per decision (<0.06% of collective latency), correct rejection of all unsafe programs, and sub-microsecond policy swaps.

1 INTRODUCTION

Large-scale GPU training depends on collective communication performance. Frameworks such as Megatron-LM [11], DeepSpeed [8], and PyTorch FSDP [14] rely on NCCL [7] for all inter-GPU data movement. The choice of algorithm (ring vs. tree), protocol (LL vs. Simple), and channel count significantly affects collective latency [3, 4]. NCCL exposes a plugin system—tuner, profiler, net, and env plugins—that allows operators to customize these decisions, and major cloud providers already ship custom tuner plugins [3].

However, every plugin executes as native code in NCCL's address space. This creates three problems:

- **No verification.** A null-pointer dereference crashes the training job; an infinite loop hangs the communicator; an off-by-one error silently corrupts NCCL internal state. In clusters where jobs run for days across thousands of GPUs, such failures waste enormous resources and are difficult to diagnose.
- **No cross-plugin state sharing.** NCCL's tuner and profiler are independent extension points with no communication mechanism. A profiler that measures collective latency cannot pass that information to a tuner that could adapt—ruling out closed-loop control.
- **No live update.** Updating a plugin requires restarting the job. Operations teams that must iterate on tuning policies across heterogeneous clusters have no way to roll out changes safely at runtime.

We observe that eBPF's execution model is a natural fit for this setting. The key insight is that NCCL's policy hot path (`getCollInfo`) runs on the CPU and is *latency-tolerant*: the GPU collective it configures takes microseconds to milliseconds, so 50–100 ns of eBPF overhead is negligible. This contrasts with eBPF's traditional domain—packet processing and syscall filtering—where nanosecond budgets are tight. In collective communication, the overhead budget is generous, and eBPF's three capabilities map directly to the three gaps above: load-time verification eliminates crash-inducing bugs, typed maps enable structured cross-plugin state sharing, and atomic program replacement provides policy hot-reload.

NCCLPol embeds bpftime [13], a userspace eBPF runtime, inside a standard NCCL plugin. Policy authors write restricted C compiled to BPF bytecode, verified at load time, and JIT-compiled to native code. The system loads via standard `NCCL_TUNER_PLUGIN` and `NCCL_PROFILER_PLUGIN` environment variables, requiring zero modifications to NCCL. This paper makes four contributions:

(1) **NCCLPol**, a system that embeds verified eBPF policy execution inside NCCL's plugin interface with no NCCL modifications (§3–4).
(2) A map-based composability model that enables closed-loop profiler-to-tuner adaptation—a capability absent from NCCL's native plugin architecture (§3).
(3) Atomic policy hot-reload with 0.774 µs swap time and zero call loss across 400,000 invocations (§3).
(4) An evaluation on NCCL 2.29.7 showing 52–100 ns CPU overhead per decision (<0.06% of collective latency) and end-to-end GPU overhead within measurement noise (§5).

2 BACKGROUND AND MOTIVATION

NCCL plugin architecture. NCCL organizes extensibility around four plugin types loaded via `dlopen`: the *tuner* plugin's `getCollInfo` selects algorithm, protocol, and channel count per collective call; the *profiler* plugin receives timestamped event callbacks; the *net* plugin controls network connections; and the *env* plugin sets runtime parameters. Major training frameworks—Megatron-LM [11], DeepSpeed [8], PyTorch FSDP [14], and JAX [2]—all depend on NCCL's collective primitives. These plugins are independent extension points with no cross-plugin communication mechanism and no verification of correctness.

The safety gap. Native plugins execute with full process privileges. A null dereference produces an immediate `SIGSEGV`, terminating the training job. An off-by-one error can silently corrupt NCCL internal state, producing subtly incorrect algorithm selections that degrade training convergence without any crash or error message—a failure mode worse than a crash because it is silent.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

An infinite loop hangs the communicator indefinitely. In all cases, the root cause is that `dlopen`-based extensions have no safety net. The situation worsens when operators must roll out plugin updates across a fleet without a mechanism for safe hot-reload.

Why eBPF fits. Three properties make eBPF well-suited: (1) a generous overhead budget—`getCollInfo()` runs on the CPU before GPU-side collectives that take microseconds to milliseconds, so 50–100 ns of eBPF overhead is negligible; (2) typed maps that provide concurrent, verified state sharing between profiler and tuner without ad-hoc shared memory; and (3) atomic program replacement for hot-reload with verification of the replacement before swap.

3 DESIGN

3.1 Design Tensions

T1: Safety vs. overhead. Verification at every invocation would add milliseconds; verification only at load time (1–5 ms, amortized over the job lifetime) trades per-call checks for a one-time cost. This relies on the invariant that verified BPF bytecode, once JIT-compiled, cannot violate its safety guarantees at runtime. The tradeoff: verification catches memory safety and termination bugs, not semantic correctness.

T2: Structured sharing vs. flexibility. All cross-component state sharing uses typed eBPF maps with atomic access semantics. Maps constrain policies to fixed-size keys/values, but NCCL policy state is inherently simple (per-communicator scalars), and structured sharing eliminates the locking bugs that plague ad-hoc native plugin state sharing.

T3: Availability vs. consistency during updates. Hot-reload atomically swaps the function pointer; any in-progress call completes under the old policy, the next call uses the new one. No call is lost. If the new policy fails verification, the old policy continues. The tradeoff: different threads may briefly execute different policies, which is acceptable because collective decisions are independent per call.

T4: Compatibility vs. capability. NCCLPol operates within NCCL's existing plugin ABI—no source modifications, no custom builds, no kernel modules. The action space is limited to algorithm, protocol, and channel count selection. Deeper interventions are outside scope, but the exposed knobs cover the most impactful tuning decisions. NCCLPol selects among NCCL's built-in algorithms; it does not implement custom collective algorithms.

3.2 Threat Model

Policies are written by platform operators, not untrusted tenants. The eBPF verifier (PREVAIL-based) checks memory safety, bounded execution, stack safety, and helper whitelisting. The system does *not* protect against resource exhaustion, side channels, bugs in the trusted computing base (bpftime JIT, PREVAIL verifier, host plugin), or semantically incorrect decisions. This provides defense in depth—eliminating crashes, hangs, and memory corruption—analogous to the Linux kernel's eBPF model.

3.3 Architecture

Figure 1 shows the architecture. Policy authors write restricted C compiled to BPF ELF objects. At load time, each program is verified (abstract interpretation over all execution paths) and JIT-compiled to x86-64 code. The map subsystem provides intra-policy state and cross-plugin communication: a profiler eBPF program writes latency observations; the tuner reads them. Hot-reload verifies and JIT-compiles a replacement, then atomically swaps the function pointer via compare-and-swap.

Policy programming model. A policy takes a `policy_context` (collective type, message size, rank count) and writes algorithm, protocol, and channel count decisions. The verifier ensures policies only read input fields and write output fields. Map lookups require null checks before dereference—the same check that prevents the most common native plugin crash (null-pointer dereference after failed lookup).

Listing 1: Size-aware tuning policy (12 lines of logic).

```
SEC("policy")
int size_aware(struct policy_context *ctx) {
  if (ctx->msg_size <= 4096) {
    ctx->algorithm = NCCL_ALGO_TREE;
    ctx->protocol = NCCL_PROTO_SIMPLE;
  } else if (ctx->msg_size <= 65536) {
    ctx->algorithm = NCCL_ALGO_TREE;
    ctx->protocol = NCCL_PROTO_LL;
  } else {
    ctx->algorithm = NCCL_ALGO_RING;
    ctx->protocol = NCCL_PROTO_LL;
  }
  return 0;
}
```

More complex policies read profiler-provided latency from eBPF maps and adapt channel count dynamically:

Listing 2: Profiler-informed channel adaptation. The `latency_map` is written by the profiler plugin and read by the tuner policy.

```
1  SEC("policy")
2  int adaptive_channels(struct policy_context *ctx) {
3    __u32 key = ctx->comm_id;
4    struct latency_state *st =
5      bpf_map_lookup_elem(&latency_map, &key);
6    if (!st) { /* first call: initialize */
7      ctx->n_channels = 8; return 0;
8    }
9    if (st->avg_latency_ns > 1000000)
10     st->channels = min(st->channels + 1, 16);
11   ctx->n_channels = st->channels;
12   return 0;
13 }
```

The mandatory null check on line 5 is enforced by the verifier; omitting it would cause load-time rejection. This closed-loop control has no built-in support in NCCL's native plugin architecture, where tuner and profiler plugins share no state mechanism.

4 IMPLEMENTATION

NCCLPol is implemented in approximately 2,500 lines of C/C++ (plugin host) and 500 lines of restricted C (policy programs).

bpftime integration. We embed bpftime [13] as the userspace eBPF runtime. bpftime provides an LLVM-based JIT compiler, a PREVAIL-based [5] static verifier, and a map subsystem. The verifier runs in the same address space as the plugin, adding approximately 1–5 ms of one-time startup overhead amortized over the lifetime of

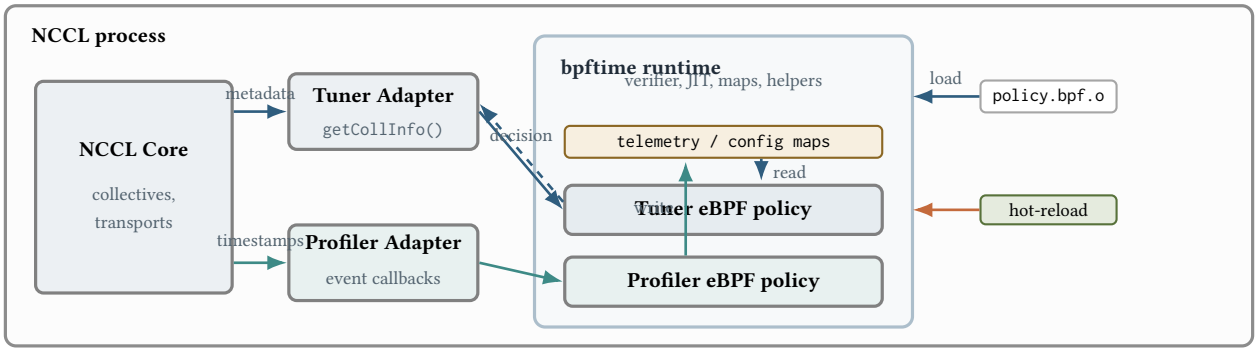

Figure 1: NCCLPol architecture. Policy programs are verified via PREVAIL, JIT-compiled by bpftime, and execute inside the NCCL process. Profiler and tuner communicate through shared typed eBPF maps. The entire system loads as standard NCCL plugins.

the training job. The LLVM JIT produces optimized x86-64 code that benefits from the same optimization passes as `clang -O2`, which is why JIT-compiled eBPF approaches native-code performance.

Plugin registration. NCCLPol registers itself as both a tuner and profiler plugin via NCCL's `ncclTunerPlugin_v3` (or v5) and `ncclProfilerPlugin_v1` (or v6) interfaces. The tuner plugin's `getCollInfo()` function constructs a `policy_context`, invokes the JIT-compiled eBPF function, and copies the output fields into NCCL's result structure. The profiler plugin's event callbacks extract latency information and update the shared eBPF maps.

NCCL integration challenges. NCCL's tuner API uses cost arrays rather than direct algorithm IDs: the tuner sets costs to zero for preferred choices and to a sentinel value (`1e9`) for others, allowing NCCL to fall back gracefully if the requested combination is unavailable. Our integration translates eBPF policy outputs into cost array entries. NCCL also passes a maximum channel count that the tuner must respect; our native baseline layer clamps the policy's request. Communicator identification required deriving a stable ID from the context pointer via hashing, since the communicator ID is not directly exposed in the tuner API as a simple integer.

Hot-reload mechanism. The active policy is stored as an atomic function pointer. Reload has three phases: (1) verify the new BPF program, (2) JIT-compile it, (3) atomic compare-and-swap on the pointer. The total reload cost is ~7.3 ms; only the final swap (0.774 µs) touches the hot path. The old pointer is retained until in-flight calls drain. If the replacement fails verification, the swap is aborted and the old policy continues—the system never enters an unverified state.

Native baseline for comparison. To provide a fair overhead comparison, we implemented a native C++ baseline that performs identical policy logic without the eBPF layer, compiled with the same optimization flags (`-O2`). The overhead difference between the native baseline and eBPF policies directly measures the cost of the eBPF dispatch and JIT execution layers, isolated from any policy logic cost.

Table 1: CPU microbenchmark (1M calls each). eBPF policies add 52–100 ns over native code.

Policy	P50 (ns)	P99 (ns)	ΔP50
Native baseline	10	16	—
`noop`	61	74	+51
`size_aware_v2`	62	73	+52
`lookup_only`	88	104	+78
`lookup_update`	113	134	+103
`adaptive_channels`	112	136	+102
`slo_enforcer`	110	134	+100

5 EVALUATION

We evaluate three research questions: (RQ1) CPU overhead and decomposition, (RQ2) verifier effectiveness and hot-reload integrity, and (RQ3) policy expressiveness within the eBPF model.

Testbed. CPU microbenchmarks run on a 24-core x86-64 platform with isolated cores; each benchmark performs 1 million calls and reports P50/P99 latencies. GPU experiments use a single NVIDIA RTX 5090 with CUDA 12.9 and NCCL 2.29.7, emulating 2 ranks over socket transport. While modest in scale, this exercises the full plugin integration path. The CPU-side policy overhead is independent of rank count.

5.1 RQ1: CPU Overhead

Table 1 reports per-call latency. Figure 2 shows the overhead staircase, which decomposes total overhead into three additive components:

- **eBPF dispatch** (+51 ns): The fixed cost of entering the JIT-compiled function from native code, including context setup, function pointer indirection, and callee-save register spill/restore. The `noop` policy isolates this cost.
- **Map lookup** (+26 ns per operation): The `lookup_only` policy performs one `bpf_map_lookup_elem` call, adding 26 ns. This cost includes hash computation on the key and hash table lookup.

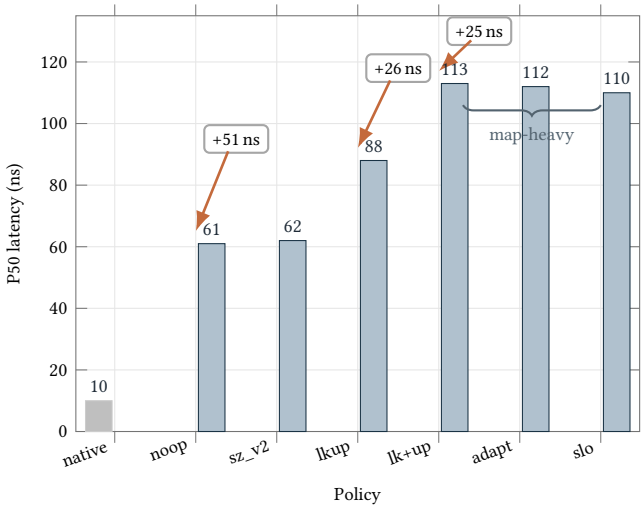

Figure 2: Overhead decomposition. The base dispatch cost (+51 ns) dominates; each map lookup adds ~26 ns and each map update adds ~25 ns.

- **Map update** (+25 ns per operation): The `lookup_update` policy adds one `bpf_map_update_elem` call, adding another 25 ns for key hashing and value copy.

The overhead is predictable: approximately $51 + 26n_{\text{lookup}} + 25n_{\text{update}}$ ns. The last three policies cluster at ~110 ns because they all perform similar numbers of map operations; the 2–3 ns differences are within noise.

P99 latencies track P50 within 20–25 ns, indicating minimal variance from JIT-compiled code with no allocation or system calls.

Context. The most complex policy adds 100 ns over native code. For a 128 MB AllReduce (175 µs on our GPU), this is 0.06% of collective latency. Even for 1 KB messages (~2 µs), the overhead is under 5%.

End-to-end GPU overhead. Figure 3 shows GPU-measured AllReduce latency across 10 message sizes (8 B to 128 MB) for four configurations: no plugin, `noop`, `size_aware_v2`, and `slo_enforcer`. All four lines are visually indistinguishable, confirming that the CPU-side 52–100 ns is invisible at the GPU level. This is because the policy decision occurs before the GPU collective launch, and NCCL's internal processing between `getCollInfo()` and kernel launch adds microseconds that dwarf the policy cost.

Limitations. These benchmarks use isolated cores on a single node with socket transport. Production deployments with NUMA effects and concurrent GPU DMA may show higher variance. The overhead formula is an empirical fit to our policies; policies with larger map values may deviate. We do not test multi-node or InfiniBand configurations.

5.2 RQ2: Safety and Hot-Reload

Table 2 shows verifier behavior on 14 programs. Each unsafe program targets a specific failure mode: the same null-dereference bug that crashes a native plugin with `SIGSEGV` is rejected at load time with an actionable error message ("must check != NULL before dereference"). The verifier tracks register types, pointer provenance, and value ranges to detect out-of-bounds access, unbounded loops, stack overflow, unauthorized helpers, input-field writes, and potential division by zero.

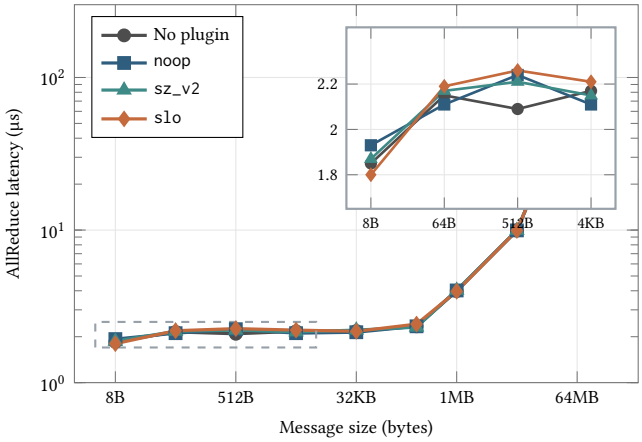

Figure 3: End-to-end AllReduce latency on RTX 5090, 2 ranks over socket transport. All four configurations are visually indistinguishable, confirming negligible GPU-level overhead.

Table 2: Verifier results: 14/14 correct verdicts. Each unsafe program targets a distinct bug class. These are author-constructed test cases.

Program	Bug Class	Result
Safe policies (7)		
`noop`, `size_aware`, `size_aware_v2`, `lookup_only`, `lookup_update`, `adaptive_ch.`, `slo_enforcer`	—	ACCEPT
Unsafe programs (7)		
`bad_lookup`	Null ptr deref	REJECT
`bad_oob_access`	OOB ctx read	REJECT
`bad_unregistered`	Illegal helper	REJECT
`bad_stack_overflow`	Stack overflow	REJECT
`bad_infinite_loop`	Unbounded loop	REJECT
`bad_write_ctx`	Write input field	REJECT
`bad_div_zero`	Division by zero	REJECT

Safety comparison. We implemented `bad_lookup` as a native C plugin. Loading it into NCCL produces:

```
Native plugin: Signal: SIGSEGV (address 0x0)
               in getCollInfo() at native_bad_plugin.so
eBPF policy:   VERIFIER REJECT: R0 is a pointer to
               map_value_or_null; must check != NULL
               before dereference at insn 7
```

This is the core safety proposition: common bugs that crash training jobs with native plugins are caught at load time with specific, actionable error messages.

Hot-reload integrity. We continuously invoke `getCollInfo()` while swapping the active policy. The atomic swap completes in 0.774 µs; the full reload (verify + JIT + swap) takes ~7.3 ms. Across 400,000 invocations, we observe zero failed or lost calls. The reload is triggered at call 200,000; the first call under the new policy occurs at call 254,709 (the intervening ~54,700 calls complete under the old policy while verification and JIT run concurrently). When we attempt to hot-reload an unsafe policy, it is rejected and the current policy continues uninterrupted. This fail-safe property means operator mistakes during updates cannot crash a running job.

Limitations. The verifier test suite is author-constructed; we do not claim coverage of all possible unsafe patterns. A real-world evaluation would benefit from a corpus of actual plugin bugs. The hot-reload test uses a single process; we do not test across ranks or under real GPU load.

5.3 RQ3: Policy Expressiveness

We demonstrate four policy capabilities beyond simple algorithm selection. We frame this as an expressiveness demonstration: we show that the eBPF programming model can express these patterns, not that the resulting parameter changes improve end-to-end training outcomes. Outcome evaluation on production workloads is future work.

Closed-loop adaptation. With the profiler plugin writing real collective latency into a shared eBPF map, the `adaptive_channels` tuner adjusts channels from 8 to 10 over successive collectives on our GPU testbed. Without profiler feedback, the same policy remains static at 8 channels. This confirms that profiler data flows through the eBPF map and affects tuner decisions—a feedback loop with no built-in support in NCCL's native plugin architecture.

Multi-tenant differentiation. The `slo_enforcer` policy serves two communicators in the same process with different SLOs: communicator 1 (SLO $\leq$ 1 ms) receives TREE / LL / 8 channels; communicator 2 (SLO $\leq$ 10 ms) receives RING / Simple / 1 channel. Per-communicator state in eBPF maps keyed by `comm_id` provides logical state separation.

Contention-aware adaptation. Figure 4 shows the adaptive channels policy's response to three contention phases over 300,000 calls. During baseline (low latency), channels ramp to 12. Under injected contention (10× latency), channels drop to 2. Upon recovery, channels return to 12. This behavior emerges from the map-based feedback loop without special contention-detection logic. We note that contention was injected synthetically; stability under real contention patterns remains to be validated.

Multi-collective coverage. NCCLPol policies work across AllReduce, AllGather, ReduceScatter, and Broadcast. AllReduce selects TREE for small messages and RING for large ones; the other collectives default to RING. A single policy expresses collective-type-aware tuning strategies covering NCCL's full range of collective operations.

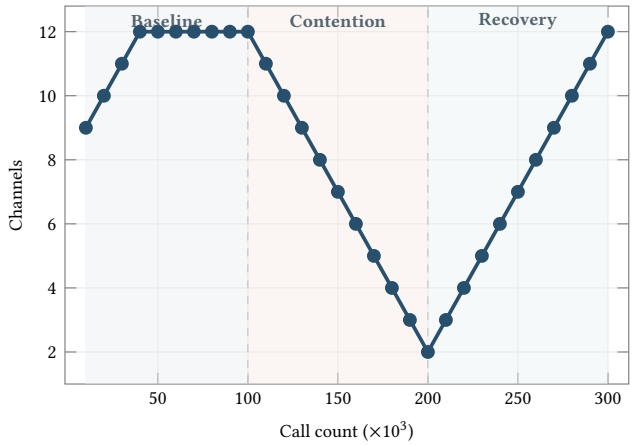

Figure 4: Channel adaptation under changing contention. The policy increases channels during low latency and decreases under contention, with symmetric recovery. Single run; contention injected via map write.

6 RELATED WORK

Collective communication. MSCCL [4] and TACCL [10] synthesize static schedules offline; they do not address runtime policy selection or safety. AutoCCL [3] automates tuning via search but uses native code without verification. MSCCL++ [9] provides GPU-driven primitives at the mechanism layer. Megatron-LM [11] and DeepSpeed [8] implement custom collective strategies at the framework level but rely on NCCL's native plugin interface without safety guarantees. NCCLPol operates at the policy layer and is complementary: a policy can activate synthesized schedules based on dynamic conditions.

eBPF for systems extensibility. XRP [15] embeds eBPF in the kernel's NVMe path, exploiting the same insight that certain system paths are latency-tolerant enough for eBPF overhead. Electrode [16] accelerates distributed protocols; eTran [6] applies eBPF to RDMA network functions; PageFlex [12] extends eBPF to page management. No prior work applies eBPF to GPU collective communication. bpftime [13] provides the userspace eBPF runtime substrate that NCCLPol builds upon.

Alternative safe-extension mechanisms. WebAssembly provides sandboxed execution but lacks eBPF's static termination and memory-safety verification; its runtime overhead is typically higher due to bounds checks that eBPF eliminates statically. Out-of-process services provide strong isolation but add microseconds of IPC latency—an order of magnitude more than NCCLPol's 52–100 ns in-process overhead. Declarative DSLs are safe by construction but cannot express stateful policies or feedback loops. Hardware isolation (Intel MPK) provides memory separation without verification; a policy can still loop indefinitely. eBPF provides both verification and efficient execution in a single model with a mature toolchain.

7 DISCUSSION AND CONCLUSION

Limitations and future work. NCCLPol currently targets NCCL's tuner and profiler interfaces; the net plugin involves complex state (RDMA queue pairs, memory registrations) that does not map cleanly to eBPF's restricted model. Our evaluation uses 2 ranks

on a single node; multi-node experiments with InfiniBand transport and large rank counts are important future work. The restricted C programming model is familiar to eBPF developers but presents a barrier for ML engineers; a higher-level policy DSL compiled to BPF would improve usability.

Not all policies can be expressed in eBPF: floating-point arithmetic, dynamic memory allocation, and GPU memory access exceed the execution model. These limitations are acceptable because policy decisions operate on metadata (message size, rank count, latency), not tensor content.

Most importantly, our evaluation demonstrates that the mechanism works but does not yet show that adaptive eBPF policies improve end-to-end training outcomes. Demonstrating benefit on production workloads at scale is the key next step.

Broader applicability. The eBPF-for-extensions pattern demonstrated by NCCLPol may be applicable beyond NCCL. AMD's RCCL [1] shares the same plugin architecture, making cross-vendor portability straightforward. Other libraries with native-code plugin interfaces—MPI implementations (OpenMPI, MPICH), distributed training frameworks [8, 14], RDMA middleware—face similar safety gaps. The observation that GPU-adjacent CPU-side decision paths have generous latency budgets for verified execution may generalize to other parts of the accelerator software stack.

Conclusion. We presented NCCLPol, a system that transforms NCCL's unsafe native plugin extension points into verified, composable policy hooks by embedding a userspace eBPF runtime. The key insight is that NCCL's CPU-side decision path is latency-tolerant enough to absorb eBPF overhead. NCCLPol adds 52–100 ns per decision (under 0.06% of collective latency), catches common crash-inducing bugs at load time, and supports atomic hot-reload (~7.3 ms total, 0.774 µs on the hot path) without call loss. The system requires zero NCCL modifications and operates entirely within the stock plugin ABI.

REFERENCES

[1] AMD. 2024. RCCL: ROCm Communication Collectives Library. https://github.com/ROCm/rccl.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. JAX: Composable Transformations of Python+NumPy Programs. https://github.com/google/jax.

[3] Zhihao Cai et al. 2024. AutoCCL: Automated Collective Communication Tuning for GPU Clusters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC).*

[4] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCL: Programmable Communication Schedules for GPU Clusters. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[5] Elazar Gershuni, Nadav Amit, et al. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI).*

[6] Bing Liu et al. 2024. eTran: Efficient and Extensible Transparent Network Functions. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[7] NVIDIA. 2024. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl.

[8] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.*

[9] Mehrdad Salehi et al. 2025. MSCCL++: A GPU-driven Communication Stack for Scalable AI Training. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys).*

[10] Aashaka Shah et al. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[11] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[12] Xueyuan Wang et al. 2025. PageFlex: eBPF-Based Programmable Page Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS).*

[13] Tong Yu, Yusheng Zheng, et al. 2025. bpftime: Userspace eBPF Runtime for Uprobe, Syscall and Kernel-Function Tracing. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[14] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Les Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Benjamin Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. In *Proceedings of the VLDB Endowment.*

[15] Yuhong Zhong, Haoyu Zhu, et al. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[16] Yu Zhou et al. 2023. Electrode: Accelerating Distributed Protocols with eBPF. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*